

Re-Estimating Home Prices in Ames Iowa



Submitted by: Jasmine (Jamie) Baldwin, Eric G. Bing &
Sickandar Akhthar

18 February 2025

Statistics 6302: Data Assignment #1

1. Introduction

Background and Context

Established in 1864, Ames Iowa has long been a commercial and agricultural hub with a strong real estate market. This underlying infrastructure and economic stability helped the Ames housing market stay relatively stable during the financial crisis of 2008 (DeCook, 2011).

Our team previously analyzed the determinants of housing prices in Ames, Iowa, using a publicly available dataset of residential property sales from 2002 to 2010. Building on the insights gained from that work, the current project refines our approach by applying Ridge, Lasso, and Elastic Net regression models within the Tidymodels pipeline. By leveraging these regularized regression techniques, we aim to improve price estimation accuracy while enhancing model interpretability and generalizability

This report is intended to be useful for executives and staff in the real estate industry.

Project Goals

Our specific goals are to:

1. Expand feature engineering, including the creation of at least 3 new variables to better capture property characteristics.
 2. Apply Ridge, Lasso, and Elastic Net regression models in order to identify the best performing model.
 3. Use cross-validation for optimal model selection, and finally,
 4. Compare performance with past models and submit results to Kaggle.
-

Feature Engineering

The dataset includes 2,930 home sales across 81 variables, capturing construction details (e.g., size, materials) and location attributes (e.g., proximity to roads, parks, and railways). Key variables relate to factors that can influence home pricing such as construction (age, size, quantity, and quality of features such as bedrooms, bathrooms, and construction materials) or location attributes (such as road and alley access, slope of land and proximity to main roads and railways).

For a complete list of variables, please see Appendix A.

Handling Missing Data and Categorical Variables

Before creating new variables or interaction terms we preprocessed the data. We removed two outliers (ID numbers 524 and 1299) from our training data that we thought might skew the predictive power. This was supported by the Influence Plot (figure 1). Next, the conversion of character columns to factors came in place. Then the removal of the Utilities variable as it only had one level and thus made no impact on the predictive power. Missing values were then imputed using k-nearest neighbors. To cope with variables with levels that were not common and thus most likely didn't contribute much to the predictive power the use of categorizing those as one level dubbed "other" took place. After the creation of the new variables both mutated and interaction terms dummy encoding, normalization, and the removal of both variables that were highly correlated and highly sparse/unbalanced took place.

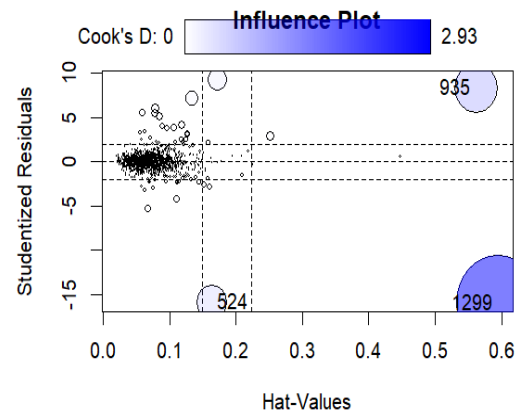


Figure 1

Newly Created Variables

Based on prior knowledge of the U.S. real estate market, literature indicating that hedonic factors influence sales price (Sirmans, Macpherson, & Zietz, 2005) and sources related to previous analyses of this dataset (Bruin, 2016) we introduced multiple composite variables to enhance the predictive accuracy of our model. After evaluating their impact, we retained the five strongest:

Total Area – A sum of the indoor and outdoor developed spaces, includes the square footage of the basement, first and second floors, garage, open and screen porches, wood deck and pools areas. *Rationale:* Since outdoor amenities such as porches and pools contribute to home valuation, incorporating them into a single metric provides a more holistic measure of overall space availability.

Total Functional Area - A sum of the areas in the first and second floors and basement. *Rationale:* The metric isolates the primary livable indoor space, which is particularly relevant in climates with harsh winters or humid summers where outdoor space usage may be limited. It provides potential buyers with direct measure of the home's practical living capacity.

Total Bathrooms – A sum of all the bathrooms in the house *Rationale:* By consolidating all bathroom counts into a single variable, we simplify model interpretation while retaining predictive power

Total House Porches – The sum of square feet for all types of porch attributes (covered, uncovered, etc.) *Rationale:* Porches contribute to curb appeal and outdoor living quality. We created a unified measure that better captures their collective influence on property value.

Age of House – The difference between the year sold and year built. *Rationale:* Home age is a key driver of depreciation and maintenance costs.

Interaction Terms

A total of 13 interaction terms were introduced to enhance the model's ability to capture complex relationships that influence property values. These interactions were primarily designed to reflect key principles of hedonic pricing (citation), which accounts for how individual property attributes combine to affect sales price. We specifically focused on interactions that capture the interplay between structural characteristics and external factors, such as:

- **Overall Quality × Neighborhood** – Buyers may be willing to pay more for a well-maintained home in an average neighborhood than for a poorly maintained home in a high-end neighborhood. This interaction reflects how property condition and location jointly influence perceived value.
- **Lot Size × House Size** – Larger lots may add more value to spacious homes than to smaller ones, reflecting differences in how lot size contributes to overall livability and appeal.

For a comprehensive list, please see Appendix C.

Handling Non-Linearity

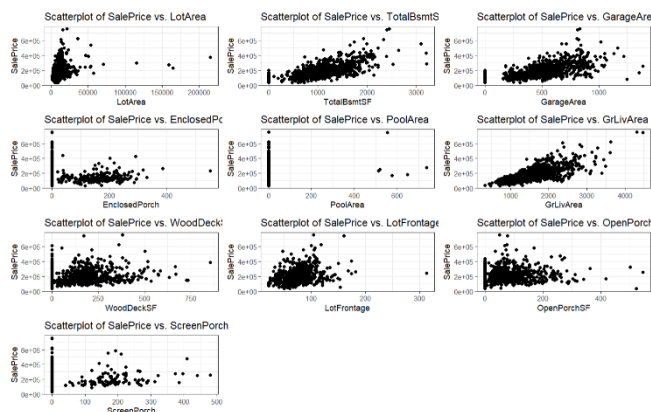


Figure 2

predictions.

LotFrontage variable using a natural cubic spline with five degrees of freedom to better reflect its relationship with SalePrice. Scatterplots showed that the effect of LotFrontage on price was not a simple straight-line pattern. By applying this smoothing technique, we were able to create a model that better accounted for how changes in LotFrontage impact property values. The LotArea variable was also adjusted to make its distribution more balanced, ensuring that properties with extremely large or small lot sizes did not disproportionately affect the model's

Identifying Low-Variance Variables

We removed features that showed very little variation across properties, as they provided minimal useful information for the model. For example, the Utilities feature was removed because nearly all properties had the same utility setup, making it irrelevant for predicting price differences.

For a full list of variables removed, see appendix A.

Development of Regression Models

We developed three regularized regression models—Ridge Regression, Lasso Regression, and Elastic Net Regression—using the tidymodels framework to assess feature selection and predictive performance.

The goal for each was to identify the optimal tuning parameters, visualize solution paths, and evaluate variable retention. See Appendix D for starting variables and retained variables for each model.

Ridge Regression

Optimal Tuning Parameter Values:

The penalty parameter (lambda) was selected based on root mean squared error (RMSE) using a grid search. The optimal value of $\lambda = 3.91\text{e-}10$ minimized RMSE.

Solution Path:

The solution path shows that as lambda increases, all coefficients shrink toward zero, but none are eliminated completely.

Resampling Technique:

We used 10-fold cross-validation with 5 repeats, ensuring a balanced trade-off between bias and variance when evaluating model performance.

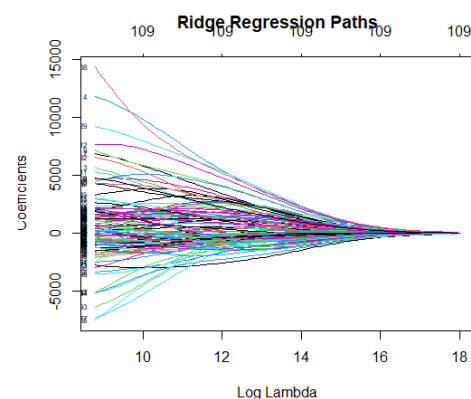


Figure 3

Lasso Regression

Optimal Tuning Parameter Values

To optimize model performance, we conducted a grid search to determine the best lambda value for regularization. Using root mean squared error (RMSE) as the evaluation metric, we identified the optimal $\lambda = 4.262041\text{e-}10$.

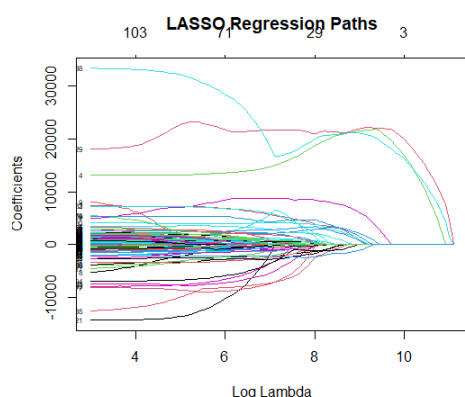


Figure 4

Solution Path

The solution path plot demonstrates how coefficient estimates change as lambda increases. As regularization strength grows, some coefficients shrink to zero, while others remain small but nonzero, indicating that the model is selectively removing less important predictors. This ability to enforce sparsity helps improve interpretability and prevents overfitting.

Resampling Technique

To ensure robust generalization, we applied 10-fold cross-validation with 5 repeats, stratifying by SalePrice to maintain distribution consistency across training folds.

Elastic Net Regression

Optimal Tuning Parameter Values

To optimize the model, we conducted a grid search that tuned both lambda (the penalty parameter) and alpha (the mixture parameter). The best combination, selected based on root mean squared error (RMSE), was $\lambda = 1.000000\text{e-}10$ and $\alpha = 0.6666667$, ensuring an optimal balance between feature selection and coefficient shrinkage.

Solution Path

The solution path plot illustrates how coefficients change as lambda and alpha vary. As regularization

strength increases, some predictors shrink to zero. Others remain but with smaller weights.

Resampling Technique

We used 10-fold cross-validation with 5 repeats, stratified by SalePrice, to ensure a balanced and reliable estimate of the model's performance. This technique helps reduce variability and provides a more robust measure of generalization performance.

Kaggle Submission

The predictions from our best-performing model (Elastic Net) were submitted to Kaggle, yielding a score of 0.14110. This score approached, but did not surpass, the top scores submitted by our team members last semester. Compared to our previous models, which included more extensive feature engineering, this iteration used 10-fold cross-validation with 5 repeats, to create more robust hyperparameter tuning and model validation. We also addressed non-linear relationships using spline transformations, allowing for more flexible modeling of complex feature effects. Finally, the Tidymodels framework improved consistency and clarity by streamlined preprocessing, model tuning, and evaluation

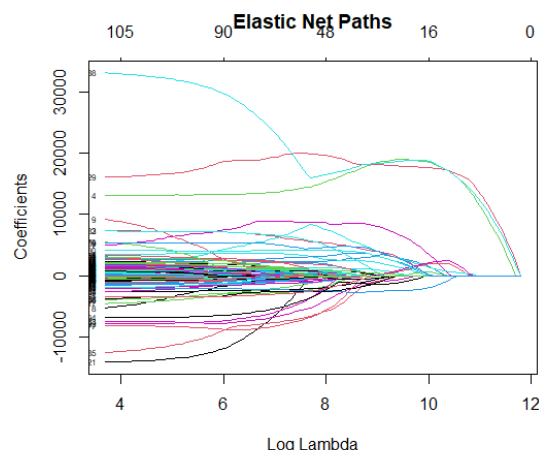


Figure 5

✓	test_predictionsEN.csv Complete · now · Final Elastic Net	0.14110
✓	test_predictionsL.csv Complete · 20m ago · Final LASSO	0.14131
✓	test_predictionsRR.csv Complete · 36m ago · Final Ridge Regression	0.14493

Figure 6

Conclusion & Limitations

Our final model used Elastic Net regression, which provided the best Kaggle score and RMSE for this project. Despite our optimization efforts, our Kaggle score did not match the lower scores achieved by our teammates last semester (0.11–0.12). We identified that interaction terms involving GarageArea × GarageCars and YearRemodel × YearBuilt had an adverse effect on our Kaggle score. Additionally, we engineered five new variables to enhance predictive accuracy. As in previous projects, our interaction terms focused on hedonic factors believed to influence housing prices. Despite repeated model tuning, our predictions resulted in only minor variations in Kaggle scores, likely due to randomized 10-fold cross-validation.

Several factors may have contributed to our slightly higher (less favorable) Kaggle score compared to past including difference in data imputation (we did not manually impute missing values for NULL columns, which may have affected model performance), tuning differences, and a different approach to feature engineering.

References

Ames History Museum, Aimes Origin, accessed 8 December 2024,

<https://ameshistory.org/content/ames-origin>

Bruin, E. (2016). *House prices: Lasso, XGBoost, and a detailed EDA*. Kaggle.

<https://www.kaggle.com/code/erikbruin/house-prices-lasso-xgboost-and-a-detailed-eda>

Communities of Excellence_2024, Ames Iowa, accessed 8 December 2024,

<https://communitiesofexcellence2026.org/what-we-do/learning-collaborative/communities/ames-iowa.html#:~:text=Community%20Definition%20and%20Identity,country's%20first%20land%20grant%20universities>.

Hayer, Saige. Aimes Neighborhoods, Iowa State Daily, August 3, 2014, accessed 8 December 2024,

<https://iowastatedaily.com/103905/special-sections-welcome-home/ames-neighborhoods/#:~:text=North%20Ames,south%20ames>

Sirmans, G. S., Macpherson, D. A., & Zietz, E. N. (2005). The composition of hedonic pricing models. *Journal of Real Estate Literature*, 13 (1),3–46.

Appendix A Dataset Description

File descriptions

- **train.csv** - the training set
- **test.csv** - the test set
- **data_description.txt** - full description of each column, originally prepared by Dean De Cock but lightly edited to match the column names used here
- **sample_submission.csv** - a benchmark submission from a linear regression on year and month of sale, lot square footage, and number of bedrooms

Baseline Variables

- **SalePrice** - the property's sale price in dollars. This is the target variable that you're trying to predict.
- **MSSubClass**: The building class
- **MSZoning**: The general zoning classification
- **LotFrontage**: Linear feet of street connected to property
- **LotArea**: Lot size in square feet
- **Street**: Type of road access
- **Alley**: Type of alley access
- **LotShape**: General shape of property
- **LandContour**: Flatness of the property
- **Utilities**: Type of utilities available
- **LotConfig**: Lot configuration
- **LandSlope**: Slope of property
- **Neighborhood**: Physical locations within Ames city limits
- **Condition1**: Proximity to main road or railroad
- **Condition2**: Proximity to main road or railroad (if a second is present)
- **BldgType**: Type of dwelling
- **HouseStyle**: Style of dwelling
- **OverallQual**: Overall material and finish quality
- **OverallCond**: Overall condition rating

- **YearBuilt:** Original construction date
- **YearRemodAdd:** Remodel date
- **RoofStyle:** Type of roof
- **RoofMatl:** Roof material
- **Exterior1st:** Exterior covering on house
- **Exterior2nd:** Exterior covering on house (if more than one material)
- **MasVnrType:** Masonry veneer type
- **MasVnrArea:** Masonry veneer area in square feet
- **ExterQual:** Exterior material quality
- **ExterCond:** Present condition of the material on the exterior
- **Foundation:** Type of foundation
- **BsmtQual:** Height of the basement
- **BsmtCond:** General condition of the basement
- **BsmtExposure:** Walkout or garden level basement walls
- **BsmtFinType1:** Quality of basement finished area
- **BsmtFinSF1:** Type 1 finished square feet
- **BsmtFinType2:** Quality of second finished area (if present)
- **BsmtFinSF2:** Type 2 finished square feet
- **BsmtUnfSF:** Unfinished square feet of basement area
- **TotalBsmtSF:** Total square feet of basement area
- **Heating:** Type of heating
- **HeatingQC:** Heating quality and condition
- **CentralAir:** Central air conditioning
- **Electrical:** Electrical system
- **1stFlrSF:** First Floor square feet
- **2ndFlrSF:** Second floor square feet
- **LowQualFinSF:** Low quality finished square feet (all floors)
- **GrLivArea:** Above grade (ground) living area square feet
- **BsmtFullBath:** Basement full bathrooms

- **BsmtHalfBath:** Basement half bathrooms
- **FullBath:** Full bathrooms above grade
- **HalfBath:** Half baths above grade
- **Bedroom:** Number of bedrooms above basement level
- **Kitchen:** Number of kitchens
- **KitchenQual:** Kitchen quality
- **TotRmsAbvGrd:** Total rooms above grade (does not include bathrooms)
- **Functional:** Home functionality rating
- **Fireplaces:** Number of fireplaces
- **FireplaceQu:** Fireplace quality
- **GarageType:** Garage location
- **GarageYrBlt:** Year garage was built
- **GarageFinish:** Interior finish of the garage
- **GarageCars:** Size of garage in car capacity
- **GarageArea:** Size of garage in square feet
- **GarageQual:** Garage quality
- **GarageCond:** Garage condition
- **PavedDrive:** Paved driveway
- **WoodDeckSF:** Wood deck area in square feet
- **OpenPorchSF:** Open porch area in square feet
- **EnclosedPorch:** Enclosed porch area in square feet
- **3SsnPorch:** Three season porch area in square feet
- **ScreenPorch:** Screen porch area in square feet
- **PoolArea:** Pool area in square feet
- **PoolQC:** Pool quality
- **Fence:** Fence quality
- **MiscFeature:** Miscellaneous feature not covered in other categories
- **MiscVal:** \$Value of miscellaneous feature
- **MoSold:** Month Sold

- **YrSold:** Year Sold
- **SaleType:** Type of sale
- **SaleCondition:** Condition of sale

Near Zero Variables Removed

The zero variance variables removed were: Street, LandContour, Utilities, LandSlope, Condition2, RoofMatl, BsmtCond, BsmtFinType2, BsmtFinSF2, Heating, LowQualFinSF, KitchenAbvGr, Functional, GarageQual, GarageCond, EnclosedPorch, X3SsnPorch, and ScreenPorch. The removal of all of them was then agreed upon.

Appendix B

Created Variables.

- **TotalArea:** $\text{TotalBsmtSF} + \text{X1stFlrSF} + \text{X2ndFlrSF} + \text{GarageArea} + \text{OpenPorchSF} + \text{ScreenPorch} + \text{WoodDeckSF} + \text{PoolArea}$
- **Total_Bathrooms:** $\text{FullBath} + 0.5 * \text{HalfBath} + \text{BsmtFullBath} + 0.5 * \text{BsmtHalfBath}$
- **AgeOfHouse:** $\text{YrSold} - \text{YearBuilt}$
- **TotalPorchSF:** $\text{OpenPorchSF} + \text{EnclosedPorch} + \text{X3SsnPorch} + \text{ScreenPorch}$
- **TotalFuctionalSF:** $\text{TotalBsmtSF} + \text{X1stFlrSF} + \text{X2ndFlrSF}$

Appendix C

Interaction Terms

- **OverallQual_x_TotalArea:** Higher quality houses with more area likely command higher prices.
- **YearBuilt_x_OverallCond:** Older houses in good condition may still command higher prices compared to newer but poorly maintained houses.
- **OverallQual_x_Neighborhood:** Higher quality houses in certain/better neighborhoods likely command higher prices.
- **LotArea_x_Neighborhood:** The value of a large lot can vary significantly depending on the neighborhood. A large lot in a highly desirable neighborhood commands a premium, whereas a large lot in a less desirable neighborhood might not add as much value.
- **AgeOfHouse_x_LotArea:** The value contribution of lot size might change based on the age of the house. For older homes, a large lot might be more valuable for potential renovations or expansions. For newer homes, the relative value increase from a larger lot might be less because the house itself is already updated.
- **AgeOfHouse_x_YearRemodAdd:** The impact of a remodel might be different depending on the original age of the house. A recent remodel on a very old house might bring it up to modern standards, drastically increasing value. A similar remodel on a relatively newer house might not have the same impact.
- **MSZoning_x_TotalArea:** Zoning regulations (MSZoning) dictate what can be built on a lot. The impact of the lot area is dependent on what is allowed.
- **BsmtQual_x_TotalBsmtSF:** The value of basement square footage is heavily influenced by its quality.

- **BsmtExposure_x_BsmtFinSF1:** A walkout basement causes finished basement space to be more valuable because it effectively adds usable living area with natural light and easy access, similar to the main floor. .
- **MSZoning_x_GrLivArea:** Similar to MSZoning:TotalArea, the value of above-ground living area (GrLivArea) is impacted by zoning. A large GrLivArea in a single-family zone has a different value than the same GrLivArea in a multi-family zone.
- **BedroomAbvGr_x_GrLivArea:** The ratio of bedrooms to living area can influence value. Too few bedrooms in a large house might be seen as inefficient. Too many bedrooms in a small house might feel cramped.
- **GarageCars_x_GrLivArea:** The number of garage spaces relative to the size of the house can be a factor. A large house with only a one-car garage might be seen as a negative.
- **Neighborhood_x_OverallQual:** The impact of overall quality (OverallQual) might be amplified in certain neighborhoods. A high-quality house in a desirable neighborhood commands a greater premium than a high-quality house in a less desirable neighborhood. The interaction term captures this synergistic effect.

Appendix D: Starting and Final Variables for Models

Regression Models Starting Variables

Original Variables:

[1]	"Id"	"MSSubClass"	"MSZoning"	"LotFrontage"	"LotArea"	"Street"	"Alley"
[8]	"LotShape"	"LandContour"	"Utilities"	"LotConfig"	"LandSlope"	"Neighborhood"	"Condition1"
[15]	"Condition2"	"BldgType"	"HouseStyle"	"OverallQual"	"OverallCond"	"YearBuilt"	"YearRemodAdd"
[22]	"RoofStyle"	"RoofMatl"	"Exterior1st"	"Exterior2nd"	"MasVnrType"	"MasVnrArea"	"ExterQual"
[29]	"ExterCond"	"Foundation"	"BsmtQual"	"BsmtCond"	"BsmtExposure"	"BsmtFinType1"	"BsmtFinSF1"
[36]	"BsmtFinType2"	"BsmtFinSF2"	"BsmtUnfSF"	"TotalBsmtSF"	"Heating"	"HeatingQC"	"CentralAir"
[43]	"Electrical"	"X1stFlrSF"	"X2ndFlrSF"	"LowQualFinSF"	"GrLivArea"	"BsmtFullBath"	"BsmtHalfBath"
[50]	"FullBath"	"HalfBath"	"BedroomAbvGr"	"KitchenAbvGr"	"KitchenQual"	"TotRmsAbvGrd"	"Functional"
[57]	"Fireplaces"	"FireplaceQu"	"GarageType"	"GarageYrBlt"	"GarageFinish"	"GarageCars"	"GarageArea"
[64]	"GarageQual"	"GarageCond"	"PavedDrive"	"WoodDeckSF"	"OpenPorchSF"	"EnclosedPorch"	"X3SsnPorch"
[71]	"ScreenPorch"	"PoolArea"	"PoolQC"	"Fence"	"MiscFeature"	"MiscVal"	"MoSold"
[78]	"YrSold"	"SaleType"	"SaleCondition"				

Ridge Regression Final Variables

--- Ridge Regression Final Model Variables ---

[1]	"(Intercept)"	"Id"	"MSSubClass"	"LotArea"
[5]	"OverallQual"	"YearRemodAdd"	"MasVnrArea"	"BsmtFinSF1"
[9]	"BsmtUnfSF"	"TotalBsmtSF"	"X1stFlrSF"	"X2ndFlrSF"
[13]	"GrLivArea"	"BsmtFullBath"	"BsmtHalfBath"	"FullBath"
[17]	"HalfBath"	"BedroomAbvGr"	"TotRmsAbvGrd"	"Fireplaces"
[21]	"GarageYrBlt"	"GarageCars"	"GarageArea"	"WoodDeckSF"
[25]	"OpenPorchSF"	"MoSold"	"YrSold"	"Total_Bathrooms"
[29]	"TotalPorchSF"	"TotalFunctionalSF"	"LotFrontage_ns_1"	"LotFrontage_ns_2"
[33]	"YearBuilt_x_OverallCond"	"AgeOfHouse_x_LotArea"	"MSZoningRM_x_TotalArea"	"BsmtQualTA_x_TotalBsmtSF"
[37]	"BsmtExposureNo_x_BsmtFinSF1"	"BedroomAbvGr_x_GrLivArea"	"GarageCars_x_GrLivArea"	"NeighborhoodNames_x_OverallQual"
[41]	"MSZoning_other"	"Alley_Pave"	"LotShape_Reg"	"LandContour_other"
[45]	"LotConfig_CulDSac"	"LotConfig_Inside"	"LandSlope_other"	"Neighborhood_Edwards"
[49]	"Neighborhood_Gilbert"	"Neighborhood_NrIdgHt"	"Neighborhood_NWAmes"	"Neighborhood_OldTown"
[53]	"Neighborhood_Sawyer"	"Neighborhood_Somerst"	"Neighborhood_other"	"Condition1_Norm"
[57]	"Condition1_other"	"BldgType_Twnhse"	"BldgType_other"	"HouseStyle_X1Story"
[61]	"HouseStyle_X2Story"	"HouseStyle_other"	"RoofStyle_Hip"	"Exterior1st_Plywood"
[65]	"Exterior1st_Wd_Sdng"	"Exterior1st_other"	"Exterior2nd_MetalSd"	"Exterior2nd_Plywood"
[69]	"Exterior2nd_VinylSd"	"Exterior2nd_Wd_Sdng"	"Exterior2nd_other"	"MasVnrType_None"
[73]	"MasVnrType_Stone"	"ExterQual_TA"	"ExterCond_TA"	"Foundation_CB1ock"
[77]	"Foundation_PCnc"	"BsmtQual_Gd"	"BsmtQual_TA"	"BsmtCond_other"
[81]	"BsmtExposure_Gd"	"BsmtExposure_Mn"	"BsmtExposure_No"	"BsmtFinType1_BLQ"
[85]	"BsmtFinType1_GLQ"	"BsmtFinType1_LwQ"	"BsmtFinType1_Rec"	"BsmtFinType1_Unf"
[89]	"BsmtFinType2_other"	"HeatingQC_Gd"	"HeatingQC_TA"	"CentralAir_Y"
[93]	"Electrical_SBrkr"	"KitchenQual_Gd"	"KitchenQual_TA"	"Functional_other"
[97]	"FireplaceQu_Gd"	"FireplaceQu_TA"	"GarageType_BuiltIn"	"GarageType_Detchd"
[101]	"GarageFinish_RFn"	"GarageFinish_Unf"	"PavedDrive_Y"	"PoolQC_Fa"
[105]	"PoolQC_Gd"	"Fence_Gdwo"	"Fence_MnPrv"	"SaleType_WD"
[109]	"SaleCondition_Normal"	"SaleCondition_Partial"		

Lasso Regression Final Variables

--- LASSO Final Model Variables ---

[1]	"(Intercept)"	"Id"	"MSSubClass"	"LotArea"	"OverallQual"	"YearRemodAdd"
[7]	"MasVnrArea"	"BsmtFinSF1"	"BsmtUnfSF"	"TotalBsmtSF"	"X1stFlrSF"	"X2ndFlrSF"
[13]	"GrLivArea"	"BsmtHalfBath"	"FullBath"	"GarageArea"	"HalfBath"	"TotRmsAbvGrd"
[19]	"Fireplaces"	"GarageYrBlt"	"GarageCars"	"TotalFunctionalSF"	"WoodDeckSF"	"OpenPorchSF"
[25]	"MoSold"	"Total_Bathrooms"	"TotalPorchSF"	"BsmtQualTA_x_TotalBsmtSF"	"LotFrontage_ns_1"	"LotFrontage_ns_2"
[31]	"YearBuilt_x_OverallCond"	"AgeOfHouse_x_LotArea"	"MSZoningRM_x_TotalArea"	"BsmtExposureNo_x_BsmtFinSF1"	"BsmtQualTA_x_TotalBsmtSF"	"bedroomAbvGr_x_GrLivArea"
[37]	"GarageCars_x_GrLivArea"	"NeighborhoodNames_x_OverallQual"	"MSZoning_other"	"Alley_Pave"	"LotShape_Reg"	"LandContour_other"
[43]	"LotConfig_CulDSac"	"LotConfig_Inside"	"LandSlope_other"	"Neighborhood_Edwards"	"Neighborhood_Gilbert"	"Neighborhood_NrIdgHt"
[49]	"Neighborhood_NWAmes"	"Neighborhood_OldTown"	"Neighborhood_Sawyer"	"Neighborhood_Somerst"	"Neighborhood_other"	"Condition1_Norm"
[55]	"Condition1_other"	"BldgType_Twnhse"	"BldgType_other"	"HouseStyle_X1Story"	"HouseStyle_X2Story"	"HouseStyle_other"
[61]	"RoofStyle_Hip"	"Exterior1st_Plywood"	"Exterior1st_Wd_Sdng"	"Exterior1st_other"	"Exterior2nd_MetalSd"	"Exterior2nd_Plywood"
[67]	"Exterior2nd_VinylSd"	"Exterior2nd_Wd_Sdng"	"Exterior2nd_other"	"MasVnrType_None"	"MasVnrType_Stone"	"ExterQual_TA"
[73]	"ExterCond_TA"	"Foundation_CB1ock"	"Foundation_PCnc"	"BsmtQual_Gd"	"BsmtQual_TA"	"BsmtCond_other"
[79]	"BsmtExposure_Gd"	"BsmtExposure_Mn"	"BsmtExposure_No"	"BsmtFinType1_BLQ"	"BsmtFinType1_GLQ"	"BsmtFinType1_LwQ"
[85]	"BsmtFinType1_Rec"	"BsmtFinType1_Unf"	"BsmtFinType2_other"	"HeatingQC_Gd"	"HeatingQC_TA"	"CentralAir_Y"
[91]	"Electrical_SBrkr"	"KitchenQual_Gd"	"KitchenQual_TA"	"Functional_other"	"FireplaceQu_TA"	"GarageType_BuiltIn"
[97]	"GarageType_Detchd"	"GarageFinish_RFn"	"GarageFinish_Unf"	"PavedDrive_Y"	"PoolQC_Fa"	"PoolQC_Gd"
[103]	"Fence_Gdwo"	"Fence_MnPrv"	"SaleType_WD"	"SaleCondition_Normal"	"SaleCondition_Partial"	

Elastic Net Regression Final Variables

--- Elastic Net Final Model Variables ---

[1]	"(Intercept)"	"Id"	"Mssubclass"	"LotArea"	"OverallQual"	"YearRemodAdd"
[7]	"MasvnrArea"	"BsmtFinsF1"	"BsmtUnfsF"	"TotalBsmtSF"	"X1stFlrSF"	"X2ndFlrSF"
[13]	"GrLivArea"	"BsmtHalfBath"	"FullBath"	"HalfBath"	"BedroomAbvGr"	"TotRmsAbvGrd"
[19]	"Fireplaces"	"GarageYrBlt"	"GarageCars"	"GarageArea"	"WoodDeckSF"	"OpenPorchSF"
[25]	"Mosold"	"YrSold"	"TotalBathrooms"	"TotalPorchSF"	"TotalFunctionalSF"	"LotFrontage_ns_1"
[31]	"LotFrontage_ns_2"	"YearBuilt_x_OverallCond"	"AgeOfHouse_x_LotArea"	"MSZoningRM_x_TotalArea"	"BsmtQualTA_x_TotalBsmtSF"	"BsmtExposureNo_x_BsmtFinsF1"
[37]	"BedroomAbvGr_x_GrLivArea"	"GarageCars_x_GrLivArea"	"NeighborhoodNames_x_OverallQual"	"MSZoning_other"	"Alley_Pave"	"LotShape_Reg"
[43]	"LandContour_other"	"LotConfig_CulDSac"	"LotConfig_Inside"	"LandSlope_other"	"Neighborhood_Edwards"	"Neighborhood_Gilbert"
[49]	"Neighborhood_NrIdght"	"Neighborhood_NwAves"	"Neighborhood_OldTown"	"Neighborhood_Sawyer"	"Neighborhood_Somerst"	"Neighborhood_other"
[55]	"Condition1_Norm"	"Condition1_other"	"BldgType_TwHse"	"BldgType_other"	"Housestyle_x1Story"	"Housestyle_x2Story"
[61]	"HouseStyle_other"	"RoofStyle_Hip"	"Exterior1st_Plywood"	"Exterior1st_wd_Sdng"	"Exterior1st_other"	"Exterior2nd_MetalSd"
[67]	"Exterior2nd_Plywood"	"Exterior2nd_VinylSd"	"Exterior2nd_wd_Sdng"	"Exterior2nd_other"	"MasvnrType_None"	"MasvnrType_Stone"
[73]	"ExterQual_TA"	"ExterCond_TA"	"Foundation_CBlock"	"Foundation_PConc"	"BsmtQual_Gd"	"BsmtQual_TA"
[79]	"BsmtCond_other"	"BsmtExposure_Gd"	"BsmtExposure_Mn"	"BsmtExposure_No"	"BsmtFintype1_GLQ"	"BsmtFintype1_LwQ"
[85]	"BsmtFintype1_Rec"	"BsmtFintype1_Unf"	"BsmtFintype2_other"	"HeatingQC_Gd"	"HeatingQC_TA"	"CentralAir_Y"
[91]	"Electrical_SBrkr"	"KitchenQual_Gd"	"KitchenQual_TA"	"Functional_other"	"FireplaceQu_TA"	"GarageType_BuiltIn"
[97]	"GarageType_Detchd"	"GarageFinish_Kfn"	"GarageFinish_Unf"	"Pavedrive_Y"	"PoolQC_Fa"	"PoolQC_Gd"
[103]	"Fence_Gdwo"	"Fence_MnPrv"	"SaleType_wd"	"SaleCondition_Normal"	"SaleCondition_Partial"	

Appendix E: R Code

Full Quarto (.qmd) script included here. Properly formatted and commented for clarity.

```
# Libraries & Pre-Processing
```

```
``{r Libraies and Pre-Processing}
```

```
# Load necessary libraries
```

```
library(glmnet)
```

```
library(tidymodels)
```

```
library(dplyr)
```

```
library(parsnip)
```

```
library(readr) # For write_csv()
```

```
# 1. Load data and preserve original IDs
```

```
sp_train_original <- read.csv('C:\\Users\\jasmi\\OneDrive - Southern Methodist University\\6302\\Data  
Assignments\\DA1\\train.csv')
```

```
sp_test_original <- read.csv("C:\\Users\\jasmi\\OneDrive - Southern Methodist University\\6302\\Data  
Assignments\\DA1\\test.csv")
```

```
# 2. Separate IDs and data for preprocessing
```

```
sp_train_ids <- sp_train_original$Id # Store training IDs
```

```
sp_test_ids <- sp_test_original$Id # Store test IDs
```

```
sp_train <- sp_train_original[, -1] # Remove ID column for training data
```

```
sp_test <- sp_test_original[, -1] # Remove ID column for test data
```

```
# Removing outliers
```

```
rows_to_remove <- c(524, 1299)
```

```
sp_train <- sp_train_original[-rows_to_remove, ]
```

```
# 3. Create sp_trainY (outcome variable)
```

```
sp_trainY <- sp_train$SalePrice
```

```
# 4. Combine training and test data *before* factor conversion
```

```
combined_data <- bind_rows(sp_train, sp_test)
```

```
# 5. Convert character columns to factors in the COMBINED data
```

```
combined_data <- combined_data %>%
```

```
  mutate(across(where(is.character), as.factor))
```

```
# 6. Split back into training and test sets
```

```
sp_train <- combined_data %>% filter(row_number() <= nrow(sp_train))
```

```
sp_test <- combined_data %>% filter(row_number() > nrow(sp_train))
```

```
# 7. Separate predictors and outcome in training data
```

```
sp_trainX <- sp_train[, -which(names(sp_train) == "SalePrice")] # All columns except SalePrice
```

```
# 8. Create and PREPARE the recipe using the COMBINED data (but fit on training only)
```

```
sp_recipe <- recipe(SalePrice ~ ., data = sp_train) %>% # Recipe uses training data
```

```
  step_rm(Utilities) %>%
```

```
  step_impute_knn(all_predictors()) %>%
```

```
  step_novel(all_nominal_predictors()) %>%
```

```
  step_other(all_nominal_predictors(), threshold = 0.05) %>%
```

```
#####PART 1#####
```

```
step_mutate(
```

```
  TotalArea = TotalBsmtSF + X1stFlrSF + X2ndFlrSF + GarageArea + OpenPorchSF + WoodDeckSF +  
  ScreenPorch + PoolArea,
```

```
  Total_Bathrooms = FullBath + 0.5 * HalfBath + BsmtFullBath + 0.5 * BsmtHalfBath,
```

```
  AgeOfHouse = as.numeric(YrSold) - as.numeric(YearBuilt),
```

TotalPorchSF = OpenPorchSF + EnclosedPorch + X3SsnPorch + ScreenPorch,

TotalFunctionalSF = TotalBsmtSF + X1stFlrSF + X2ndFlrSF

) %>%

#Total Bath is from Bruin AgeOfHouse is mentioned in it too but we also did it in my groups project last semester

step_ns(LotFrontage, options = list(df = 5)) %>% # natural spline

step_BoxCox(LotArea) %>%

#####PART 1#####

step_interact(terms = ~ OverallQual:TotalArea) %>%

step_interact(terms = ~ YearBuilt:OverallCond) %>%

step_interact(terms = ~ OverallQual:Neighborhood) %>%

step_interact(terms = ~ LotArea:Neighborhood) %>%

step_interact(terms = ~ AgeOfHouse:LotArea) %>%

step_interact(terms = ~ AgeOfHouse:YearRemodAdd) %>%

step_interact(terms = ~ MSZoning:TotalArea) %>%

step_interact(terms = ~ BsmtQual:TotalBsmtSF) %>%

step_interact(terms = ~ BsmtExposure:BsmFinSF1) %>%

step_interact(terms = ~ MSZoning:GrLivArea) %>%

step_interact(terms = ~ BedroomAbvGr:GrLivArea) %>%

step_interact(terms = ~ GarageCars:GrLivArea) %>%

step_interact(terms = ~ Neighborhood:OverallQual) %>%

step_dummy(all_nominal_predictors()) %>%

step_zv(all_predictors()) %>% # removes variables with a single value

step_normalize(all_numeric_predictors()) %>%

step_nzv(all_predictors()) %>% # will potentially remove variables that are highly sparse and unbalanced.

step_corr(all_numeric_predictors())

sp_recipe_prep <- prep(sp_recipe)

```
# 9. Bake the training data
```

```
sp_train_baked <- bake(sp_recipe_prep, new_data = sp_train)
```

```
# 10. Bake the test data
```

```
sp_test_baked <- bake(sp_recipe_prep, new_data = sp_test)
```

```
...
```

```
# Ridge Regression
```

```
```{r Ridge Regression}
```

```
11. Model specification (RIDGE REGRESSION - alpha = 0)
```

```
sp_glmnet <- parsnip::linear_reg(
```

```
 penalty = tune(),
```

```
 mixture = 0 # Ridge regression: mixture = 0
```

```
) %>%
```

```
 set_mode("regression") %>%
```

```
 set_engine("glmnet")
```

```
12. Workflow
```

```
sp_wf <- workflow() %>%
```

```
 add_recipe(sp_recipe) %>%
```

```
 add_model(sp_glmnet)
```

```
13. 10-fold cross-validation with 5 repeats (using the *original* training data)
```

```
sp_kfolds <- vfold_cv(sp_train, v = 10, repeats = 5, strata = "SalePrice")
```

```
14. Tune the model
```

```
sp_tune <- tune_grid(
```

```
 sp_wf,
```

```
 resamples = sp_kfolds,
```

```
 grid = 10, # You can adjust the grid size as needed
```

```
metrics = metric_set(rmse, rsq, mae)
)

15. Display best results
sp_tune %>% show_best(metric = "rmse")
sp_tune %>% show_best(metric = "mae")

Display best results (and get tuning parameter values)
best_rmse <- sp_tune %>% show_best(metric = "rmse")
print("Best RMSE Results:")
print(best_rmse)

best_mae <- sp_tune %>% show_best(metric = "mae")
print("Best MAE Results:")
print(best_mae)

16. Select best tuning parameters
sp_best_tune <- sp_tune %>% select_best(metric = "rmse")

Extract the optimal penalty value
optimal_penalty <- sp_best_tune$penalty
print(paste("Optimal Penalty (lambda):", optimal_penalty))

17. Finalize workflow
sp_final_workflow <- sp_wf %>%
 finalize_workflow(sp_best_tune)

18. Fit the final model to the *original* training data
sp_final_fit <- fit(sp_final_workflow, data = sp_train)
```



```
19. Extract coefficients
```

```
sp_coef_Ridge <- sp_final_fit %>%
 fit(sp_train) %>%
 extract_fit_parsnip() %>%
 tidy()
```

```
sp_coef_Ridge
```

```
20. Make predictions on the *baked* test data
```

```
test_predictions <- predict(sp_final_fit, new_data = sp_test)
```

```
21. Rename the prediction column
```

```
test_predictions <- test_predictions %>%
 rename(SalePrice = .pred)
```

```
22. Combine IDs and predictions
```

```
test_predictions_with_ids <- tibble(Id = sp_test_ids, SalePrice = test_predictions$SalePrice)
```

```
23. Save predictions with IDs
```

```
write_csv(test_predictions_with_ids, "C:\\Users\\jasmi\\OneDrive - Southern Methodist
University\\6302\\Data Assignments\\DA1\\test_predictionsRR.csv")
```

```
...
```

```
LASSO
```

```
```{r LASSO}
```

```
# 11. Model specification (LASSO - alpha = 1)
```

```
sp_glmnet <- parsnip::linear_reg(  
  penalty = tune(),  
  mixture = 1 # LASSO regression: mixture = 1
```

```
) %>%  
  set_mode("regression") %>%  
  set_engine("glmnet")  
  
# 12. Workflow  
sp_wf <- workflow() %>%  
  add_recipe(sp_recipe) %>%  
  add_model(sp_glmnet)  
  
# 13. 10-fold cross-validation with 5 repeats (using the *original* training data)  
sp_kfolds <- vfold_cv(sp_train, v = 10, repeats = 5, strata = "SalePrice")  
  
# 14. Tune the model  
sp_tune <- tune_grid(  
  sp_wf,  
  resamples = sp_kfolds,  
  grid = 10, # You can adjust the grid size as needed  
  metrics = metric_set(rmse, rsq, mae)  
)  
  
# 15. Display best results  
sp_tune %>% show_best(metric = "rmse")  
sp_tune %>% show_best(metric = "mae")  
  
# Display best results (and get tuning parameter values)  
best_rmse <- sp_tune %>% show_best(metric = "rmse")  
print("Best RMSE Results:")  
print(best_rmse)  
  
# 16. Select best tuning parameters
```

```
sp_best_tune <- sp_tune %>% select_best(metric = "rmse")
```

```
# 17. Finalize workflow
```

```
sp_final_workflow <- sp_wf %>%  
  finalize_workflow(sp_best_tune)
```

```
# 18. Fit the final model to the *original* training data
```

```
sp_final_fit <- fit(sp_final_workflow, data = sp_train)
```

```
# 19. Extract coefficients
```

```
sp_coef_LASSO <- sp_final_fit %>%  
  extract_fit_parsnip() %>%  
  tidy()
```

```
sp_coef_LASSO
```

```
# 20. Make predictions on the *baked* test data
```

```
test_predictions <- predict(sp_final_fit, new_data = sp_test)
```

```
# 21. Rename the prediction column
```

```
test_predictions <- test_predictions %>%  
  rename(SalePrice = .pred)
```

```
# 22. Combine IDs and predictions
```

```
test_predictions_with_ids <- tibble(Id = sp_test_ids, SalePrice = test_predictions$SalePrice)
```

```
# 23. Save predictions with IDs
```

```
write_csv(test_predictions_with_ids, "C:\\Users\\jasmi\\OneDrive - Southern Methodist  
University\\6302\\Data Assignments\\DA1\\test_predictionsL.csv")
```

```
...
```

```
# Elastic Net
```

```
```{r Elastic Net}
```

```
11. Model specification (ELASTIC NET REGRESSION)
```

```
sp_glmnet <- parsnip::linear_reg(
 penalty = tune(),
 mixture = tune() # Tune the mixture as well
) %>%
set_mode("regression") %>%
set_engine("glmnet")
```

```
12. Workflow (no changes)
```

```
sp_wf <- workflow() %>%
add_recipe(sp_recipe) %>%
add_model(sp_glmnet)
```

```
13. 10-fold cross-validation with 5 repeats
```

```
sp_kfolds <- vfold_cv(sp_train, v = 10, repeats = 5, strata = "SalePrice")
```

```
14. Tune the model (important change here!)
```

```
sp_tune <- tune_grid(
 sp_wf,
 resamples = sp_kfolds,
 grid = grid_regular(penalty(range = c(-10, 0)), # Example penalty range
 mixture(range = c(0, 1)), # Mixture range 0 to 1
 levels = 10), # Adjust levels as needed
 metrics = metric_set(rmse, rsq, mae)
)
```

# 15. Display best results

```
sp_tune %>% show_best(metric = "rmse")
```

```
sp_tune %>% show_best(metric = "mae")
```

# 16. Select best tuning parameters

```
sp_best_tune <- sp_tune %>% select_best(metric = "rmse")
```

# 17. Finalize workflow

```
sp_final_workflow <- sp_wf %>%
```

```
 finalize_workflow(sp_best_tune)
```

# 18. Fit the final model to the *\*original\** training data

```
sp_final_fit <- fit(sp_final_workflow, data = sp_train)
```

# 19. Extract coefficients

```
sp_coef_EN <- sp_final_fit %>%
```

```
 extract_fit_parsnip() %>%
```

```
 tidy()
```

```
sp_coef_EN
```

# 20. Make predictions on the *\*baked\** test data

```
test_predictions <- predict(sp_final_fit, new_data = sp_test)
```

# 21. Rename the prediction column

```
test_predictions <- test_predictions %>%
```

```
 rename(SalePrice = .pred)
```

# 22. Combine IDs and predictions

```
test_predictions_with_ids <- tibble(Id = sp_test_ids, SalePrice = test_predictions$SalePrice)
```

```
23. Save predictions with IDs
```

```
write_csv(test_predictions_with_ids, "C:\\Users\\jasmi\\OneDrive - Southern Methodist University\\6302\\Data Assignments\\DA1\\test_predictionsEN.csv")
```

```
``
```

```
Data Assignment Questions and extras
```

```
``{r DA Steps}
```

```
library(ggplot2)
```

```
library(mgcv)
```

```
library(caret)
```

```
#Part 2
```

```
nearZeroVar(sp_train)
```

```
colnames(sp_train)[nearZeroVar(sp_train)]
```

```
#Part 3
```

```
Method 2: Using names() and sapply() (base R)
```

```
continuous_vars <- names(sp_train)[sapply(sp_train, is.numeric)]
```

```
print(continuous_vars)
```

```
plot_variable <- function(data, variable_name) {
```

```
 plot <- ggplot(data, aes(x = .data[[variable_name]], y = SalePrice)) +
```

```
 geom_point() +
```

```
 labs(title = paste("Scatterplot of SalePrice vs.", variable_name),
```

```
 x = variable_name,
```

```
 y = "SalePrice") +
```

```
 theme_bw()
```

```
 print(plot)
```

```
}
```

```
p1 <- plot_variable(sp_train, "LotArea")
p2 <- plot_variable(sp_train, "TotalBsmtSF")
p3 <- plot_variable(sp_train, "GarageArea")
p4 <- plot_variable(sp_train, "EnclosedPorch")
p5 <- plot_variable(sp_train, "PoolArea")
p6 <- plot_variable(sp_train, "GrLivArea")
p7 <- plot_variable(sp_train, "WoodDeckSF")
p8 <- plot_variable(sp_train, "LotFrontage")
p9 <- plot_variable(sp_train, "OpenPorchSF")
p10 <- plot_variable(sp_train, "ScreenPorch")
LotFrontage seems the least linear
```

```
library(ggpubr)
```

```
Arrange plots in a 3x4 grid
ggarrange(p1, p2, p3, p4, p5, p6, p7, p8, p9, p10,
 ncol = 3, nrow = 4)
```

```
Part 4 D
```

```
--- Variable Tracking and Summary Function ---
```

```
track_and_summarize_variables <- function(original_data, baked_data, model_coef, model_name) {
```

```
 # 1. Original Variables (from sp_train_original)
```

```
 original_variables <- names(original_data[, -which(names(original_data) == "SalePrice")])
```

```
 # 2. Variables After Baking (from sp_train_baked)
```

```
baked_variables <- names(baked_data[, -which(names(baked_data) == "SalePrice")])
```

```
3. Variables with Coefficients (from model_coef)
```

```
coef_variables <- model_coef$term # or model_coef %>% pull(term)
```

```
4. Variables Retained in the Model (coefficients not equal to zero)
```

```
retained_vars <- model_coef %>%
```

```
 filter(estimate != 0) %>%
```

```
 pull(term)
```

```
5. Variables Removed During Preprocessing (Baked - Original)
```

```
removed_preprocessing <- setdiff(original_variables, baked_variables)
```

```
6. Variables Removed by the Model (Baked - Retained)
```

```
removed_model <- setdiff(baked_variables, retained_vars)
```

```
cat("\n---", model_name, "Variable Summary ---\n")
```

```
cat("Original Variables:\n")
```

```
print(original_variables)
```

```
cat("\nVariables After Preprocessing (Baked):\n")
```

```
print(baked_variables)
```

```
cat("\nVariables with Coefficients:\n")
```

```
print(coef_variables)
```

```
cat("\nVariables Retained by", model_name, ":\n")
```

```
print(retained_vars)
```

```
cat("\nVariables Removed During Preprocessing:\n")
```

```
print(removed_preprocessing)
```

```
cat("\nVariables Removed by", model_name, ":\n")
```

```
print(removed_model)
```

```
cat("-----\n")
```



```

 return(list(original = original_variables, baked = baked_variables, coef = coef_variables, retained =
retained_vars, removed_preprocessing = removed_preprocessing, removed_model = removed_model))
}

```

```

--- Ridge Regression Variable Tracking ---

```

```

ridge_summary <- track_and_summarize_variables(sp_train_original, sp_train_baked, sp_coef_Ridge,
"Ridge Regression")

```

```

--- LASSO Variable Tracking ---

```

```

lasso_summary <- track_and_summarize_variables(sp_train_original, sp_train_baked, sp_coef_LASSO,
"LASSO") # Replace sp_coef_lasso

```

```

--- Elastic Net Variable Tracking ---

```

```

en_summary <- track_and_summarize_variables(sp_train_original, sp_train_baked, sp_coef_EN, "Elastic
Net") # Replace sp_coef_en

```

```

track_final_model_variables <- function(model_coef, model_name) {

```

```

 # Extract variables with nonzero coefficients (final model variables)

```

```

 final_vars <- model_coef %>%

```

```

 filter(estimate != 0) %>%

```

```

 pull(term)

```

```

 cat("\n---", model_name, "Final Model Variables ---\n")

```

```

 print(final_vars)

```

```

 cat("-----\n")

```

```

 return(final_vars)

```

```

}

```

```

--- Ridge Regression Final Variables ---

```

```
ridge_final_vars <- track_final_model_variables(sp_coef_Ridge, "Ridge Regression")

--- LASSO Final Variables ---
lasso_final_vars <- track_final_model_variables(sp_coef_LASSO, "LASSO")

--- Elastic Net Final Variables ---
en_final_vars <- track_final_model_variables(sp_coef_EN, "Elastic Net")

library(tidymodels)
library(car) # For influence plot

Extract nonzero coefficients from Elastic Net model
sp_coef_EN <- sp_final_fit %>%
 extract_fit_parsnip() %>%
 tidy()

Get significant predictors (excluding intercept)
important_vars <- sp_coef_EN %>%
 filter(estimate != 0, term != "(Intercept)") %>%
 pull(term)

Create a formula for a linear model using only selected predictors
formula_lm <- as.formula(paste("SalePrice ~", paste(important_vars, collapse = " + ")))

Fit an equivalent linear model (lm) using selected variables
final_model_lm <- lm(formula_lm, data = sp_train_baked)

Influence plot using car package
influencePlot(final_model_lm, id.method = "identify", main = "Influence Plot")
```

```
Identify influential points based on Cook's Distance
influential_points <- which(cooks.distance(final_model_lm) > (4 / nrow(sp_train)))
print("Influential Points Identified by Cook's Distance:")
print(influential_points)
```

```
Diagnostics: Leverage and studentized residuals
leverage <- hatvalues(final_model_lm)
studentized_residuals <- rstudent(final_model_lm)
```

```
Check for high-leverage points
high_leverage_points <- which(leverage > (2 * mean(leverage)))
print("High Leverage Points:")
print(high_leverage_points)
```

```
#####PART 4B #####
```

```
library(ggplot2)
library(tidymodels)
library(glmnet)
```

```
1. Convert training data to a matrix format for glmnet
x_train <- as.matrix(select(sp_train_baked, -SalePrice))
y_train <- sp_train_baked$SalePrice
```

```
2. Fit Ridge (alpha = 0), LASSO (alpha = 1), and Elastic Net (alpha = 0.5)
ridge_fit <- glmnet(x_train, y_train, alpha = 0) # Ridge
lasso_fit <- glmnet(x_train, y_train, alpha = 1) # LASSO
en_fit <- glmnet(x_train, y_train, alpha = 0.5) # Elastic Net
```

```
3. Plot coefficient paths
```

```
par(mfrow = c(1, 3)) # Arrange plots in a row
```

```
plot(ridge_fit, xvar = "lambda", label = TRUE, main = "Ridge Regression Paths")
```

```
plot(lasso_fit, xvar = "lambda", label = TRUE, main = "LASSO Regression Paths")
```

```
plot(en_fit, xvar = "lambda", label = TRUE, main = "Elastic Net Paths")
```

```
par(mfrow = c(1,1)) # Reset plotting layout
```

```
...
```